F I N A I

FinAI

An AI-Driven Personal Finance, Investment Analytics and Financial Intelligence Platform

M.Tech Dissertation Project

Phase 1: Foundation & Core Financial Management

Internal Review 1 - Documentation

September 2026

Executive Summary

FinAI is a full-stack web platform that unifies personal expense management, budget intelligence, statistical analytics and explainable AI-generated insight under one secure, disclaimer-governed system. This document presents the complete Phase 1 engineering record: problem statement and research gap, layered architecture and technology-stack justification, database and API design, security architecture, the AI/ML methodology with explicit formulas and guards, implementation walkthrough with interface captures, the testing record (47 passing tests plus a scripted end-to-end journey), containerised deployment, and the four-phase roadmap through which investment analytics, portfolio intelligence and a retrieval-augmented research assistant will be layered onto the delivered foundation.

The defining engineering commitment is explainability with integrity: every insight cites the aggregated data that produced it, the anomaly detector reports its reference statistics, and the financial health score exposes every factor's weight and contribution. Where data is insufficient, the system reports that fact instead of inventing output - a discipline the dissertation carries into all later phases.

Table of Contents

Chapter 1 Introduction **2**

1.1 Background 2

1.2 Problem Statement 2

1.3 Objectives 2

1.4 Scope of Phase 1 3

1.5 Report Organization 3

Chapter 2 Literature Survey and Research Gap **4**

2.1 Existing Systems and Their Limitations 4

2.2 Research Gap 4

2.3 Positioning of the Proposed Work 5

Chapter 3 Proposed System **5**

3.1 System Overview 5

3.2 Functional Requirements (Phase 1) 5

3.3 Non-Functional Requirements 7

3.4 Design Principles 7

Chapter 4 System Architecture and Technology Stack **8**

4.1 Layered Architecture 8

4.2 Technology Stack and Justification 8

4.3 Key Architecture Decisions 9

Chapter 5 Database Design **9**

5.1 Entity Overview 9

5.2 Integrity and Indexing 10

5.3 Design Decisions 10

Chapter 6 API Design **11**

6.1 Conventions 11

6.2 Rate Limiting 11

6.3 Endpoint Groups....12
Chapter 7 Security Architecture....12
7.1 Authentication....12
7.2 Authorization and Least Privilege....13
7.3 Input Handling and Transport Headers....13
7.4 Audit Trail....13
7.5 Known Phase 1 Limitations....14
Chapter 8 AI/ML Methodology....14
8.1 Principles....14
8.2 Anomaly Detection (Z-Score)....14
8.3 Guarded Insight Rule Engine....15
8.4 Explainable Financial Health Score....15
8.5 Planned Experiments (Phase 2-3 Evaluation Plan)....16
Chapter 9 Implementation....17
9.1 Repository Layout....17
9.2 Backend Implementation....17
9.3 Frontend Implementation....17
9.4 Interface Walkthrough....18
Chapter 10 Testing and Results....20
10.1 Test Strategy....20
10.2 Test Inventory and Outcome....21
10.3 End-to-End Verification....21
10.4 Observations....21
Chapter 11 Deployment....22
11.1 Development Setup....22
11.2 Containerised Stack....22
11.3 Operational Behaviour....23
Chapter 12 Limitations and Future Work....23
12.1 Current Limitations....23

12.2 Phase 2 - Investment Analytics.....23
12.3 Phase 3 - Portfolio Intelligence and Financial Planning.....24
12.4 Phase 4 - Intelligent Financial Platform.....24
Chapter 13 Conclusion.....24
References.....25
Appendix A REST API Catalogue (Phase 1).....26
Appendix B Risk Register.....27
Appendix C Four-Phase Roadmap and Review Plan.....27

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

Chapter 1 Introduction

1.1 Background

Personal finance management and investment analysis have traditionally been handled by disconnected tools: expense trackers that record spending without explaining it, spreadsheet portfolios that demand manual maintenance, and stock research platforms that present raw numbers without interpretation or transparency. With the maturity of machine learning and natural language processing, it is now feasible to build a unified platform that not only records financial data but analyses it, detects anomalies, generates grounded insights, and explains every conclusion it draws.

FinAI (An AI-Driven Personal Finance, Investment Analytics and Financial Intelligence Platform) is a full-stack web application developed as an M.Tech dissertation project. It unifies personal expense management, budget tracking, statistical analytics, AI-assisted insight generation and an explainable financial health score in one secure, production-quality platform. Finance is the application domain; software engineering, AI/ML and data engineering are the academic focus.

1.2 Problem Statement

Conventional personal finance applications suffer from three structural deficiencies. First, they are passive: they display what the user spent but never explain why it matters, what changed, or what to do next. Second, their scoring mechanisms, where present, are black boxes - users cannot see how a rating was computed or which factor contributed what. Third, they do not integrate behavioural spending data with investment analysis, leaving users to correlate the two themselves. There is a need for an integrated, explainable, data-grounded financial intelligence platform that treats analytical output as a first-class feature rather than an afterthought.

1.3 Objectives

The dissertation objectives, phased across three semesters, are:

- Design and implement a secure, layered full-stack architecture (API, business logic, analysis engine, data access) with role-based access control and versioned risk-disclaimer governance.
- Implement core financial management: expense and income recording, custom categories, budgets with threshold alerts, and multi-granularity analytics.

- Build an AI layer that is statistically grounded and explainable: z-score anomaly detection, a guarded insight rule engine whose every output cites its evidence, and a transparent weighted financial health score.
- Phase 2-4 roadmap: investment analytics (valuation engine, news sentiment), portfolio intelligence (risk metrics, stress testing, planning calculators) and a retrieval-augmented research assistant.
- Evaluate all models with measurable metrics (precision, recall, F1, MAE/RMSE/MAPE) rather than unsupported accuracy claims.

1.4 Scope of Phase 1

This documentation covers Phase 1 - Foundation and Core Financial Management - the first of four planned increments. Phase 1 delivers the complete platform skeleton (authentication, RBAC, audit trail, disclaimer gate), expense/income/budget management, the analytics engine, the guarded AI insight engine, the explainable health score, an administrative console, and a verified test suite. Investment analytics begins in Phase 2; accordingly, no financial-market data is fabricated anywhere in the current build.

1.5 Report Organization

Chapter 2 surveys existing systems and the research gap. Chapter 3 specifies the proposed system. Chapters 4-7 document architecture, database, API and security design. Chapter 8 details the AI/ML methodology with formulas. Chapter 9 walks through the implementation with annotated interface captures. Chapter 10 reports testing and results, Chapter 11 deployment, and Chapter 12 limitations and the phase roadmap. Chapter 13 concludes.

Chapter 2 Literature Survey and Research Gap

2.1 Existing Systems and Their Limitations

A survey of representative personal finance and investment tools was conducted to establish the baseline. The findings are summarised below.

System	Core Capability	Key Limitations
Spreadsheet budgets	Fully manual tracking	No automation, no anomaly detection, error-prone, no aggregation or trends
Bank/wallet apps	Automatic transaction capture	Recording only; categorisation is shallow; no explanation, scoring or budget intelligence

System	Core Capability	Key Limitations
Expense trackers (general)	Categorised spending views	Insight-free: charts without interpretation; no data-sufficiency guards; no health scoring
Investment trackers	Holdings and returns	Detached from daily spending; opaque metrics; no valuation or sentiment context
Robo-advisory tools	Allocated portfolios	Black-box recommendations; no explainability; advice orientation rather than education

2.2 Research Gap

Synthesising the survey, five gaps recur across existing tools, and each maps to a concrete FinAI response:

#	Gap in Existing Tools	FinAI Answer
1	Expense tracking without explanation	Guarded insight engine; every insight cites the aggregated figures it was derived from (Chapter 8)
2	Black-box scores	Weighted, documented health score with per-factor contribution and weight redistribution (Chapter 8)
3	No integrated investment research	Provider-abstraction based stock analyzer, valuation engine and sentiment pipeline (Phase 2)
4	AI without grounding	Retrieval-augmented assistant with mandatory citations; refuses when retrieval is empty (Phase 4)
5	Planning divorced from behaviour	Goals and calculators planned to consume the user's actual expense history (Phase 3)

2.3 Positioning of the Proposed Work

FinAI deliberately positions itself as an analytical and educational system, not an advisory one. The platform states plainly that it is not a SEBI-registered investment adviser, provides no futures-and-options recommendations (explicitly out of scope), and stores a versioned record of every user's disclaimer acceptance before the application becomes usable. This compliance-first framing distinguishes the work from advisory-oriented commercial systems.

Chapter 3 Proposed System

3.1 System Overview

FinAI is a single-page application backed by a versioned REST API. Users first confront an investment risk disclaimer and must explicitly acknowledge it; the acceptance (version, IP address, timestamp) is persisted. They then register with an email or a mobile number, authenticate via short-lived JWT access tokens and rotating refresh tokens, and land on a dashboard that unifies financial, expense, budget and AI-insight snapshots. An admin role - reachable only through the same hardened login - sees operational metrics exclusively and can never inspect a user's financial records.

3.2 Functional Requirements (Phase 1)

- **FR-1 -** Registration with email or Indian mobile number, login, logout, refresh-token rotation, password change and reset, active-session listing and revocation.
- **FR-2 -** Versioned disclaimer acceptance recorded before first use; login refused until the current version is accepted.
- **FR-3 -** Expense CRUD with category, subcategory, payment method, merchant, tags, recurring flag, date filtering, free-text search and pagination; soft deletion.
- **FR-4 -** Income recording with source and recurrence.
- **FR-5 -** Monthly or yearly budgets, overall or per category, with customisable alert thresholds and live utilisation status.
- **FR-6 -** Analytics: period summaries, monthly trend series, month-over-month deltas, category distribution, top merchants.
- **FR-7 -** AI module: anomaly flags with reference statistics, guarded insights, explainable health score.
- **FR-8 -** Admin console: platform overview, DAU/MAU, feature usage, user management, audit trail, login-activity timeline.

3.3 Non-Functional Requirements

Attribute	Requirement	Mechanism
Security	No plaintext credentials; brute-force resistance	bcrypt hashing, JWT + rotating refresh tokens, lockout, rate limiting, audit log
Privacy	Strict user data isolation; admin least privilege	Per-user query scoping; admin endpoints expose counts only
Reliability	Container dependencies may restart independently	Startup waits for database readiness (30 retries); health endpoint
Performance	Responsive dashboards on bounded windows	Composite indexes, bounded aggregation windows, pagination
Maintainability	Layered, testable design	Pure analysis engine, service layer, 47 automated tests
Portability	Zero-setup local run and production-shaped deployment	SQLAlchemy dialect portability; Docker Compose stack

3.4 Design Principles

- **No fabrication -** Every generated statement must cite the data behind it; the system stays silent when data is insufficient.
- **Explainability -** Scores decompose into weighted factors with visible contributions; nothing is hidden behind a single number.
- **Provider abstraction -** External data sources sit behind swappable interfaces so no provider is load-bearing (Phase 2).
- **Incremental delivery -** Every phase ships a usable, tested increment that can be demonstrated end to end.

Chapter 4 System Architecture and Technology Stack

4.1 Layered Architecture

The backend follows a strict four-layer discipline. Each layer has a single responsibility and a forbidden direction of dependency: routes never contain business rules, services never touch HTTP, and the analysis engine is pure computation with no I/O, which is what makes it unit-testable in isolation.

Layer	Responsibility	Must Not
API routes	HTTP parsing, authentication, status codes, rate limiting	Contain business logic
Services	Business rules and cross-entity orchestration	Use HTTP constructs
Analysis engine	Pure computation: aggregation, anomaly, insights, scoring	Perform any I/O
Data access	SQLAlchemy models and session handling	Embed business rules

The request path for analytics illustrates the flow: a route authenticates the caller and applies rate limits, the service fetches that user's rows within a bounded window, and the pure engine computes series, flags, insights and scores before the response is serialised. A public health endpoint reports liveness including a database round trip, and the container startup sequence waits for database readiness so dependency restarts cannot crash the application.

4.2 Technology Stack and Justification

Concern	Choice	Justification
API framework	FastAPI (Python)	Native async, Pydantic validation, automatic OpenAPI; first-class fit with the Python ML ecosystem used in later phases
Data validation	Pydantic v2	Typed, fast, single source of truth for API contracts
ORM / database	SQLAlchemy 2.0; PostgreSQL (prod) / SQLite (dev)	Typed Mapped models; identical schema across dialects enables zero-setup local development
Authentication	JWT access (30 min) + rotating refresh (14 days), bcrypt	Stateless API with revocable persisted sessions; digests stored, never raw tokens
Frontend	React 18 + TypeScript + Vite	Authenticated analytics SPA; SSR adds cost without benefit; fast HMR and clean builds

Concern	Choice	Justification
Styling / charts	Tailwind CSS 3; Recharts	Consistent custom glassmorphism design system; lightweight React-native chart composition
Testing	pytest + FastAPI TestClient	Isolated database per test; public-API-driven integration tests
Packaging	Docker Compose	Production-shaped stack: PostgreSQL 16, Redis 7 (Phase 2+), API, nginx-served web build

4.3 Key Architecture Decisions

- SQLite for development, PostgreSQL for production through one SQLAlchemy schema (ADR-001) - a demonstration must run on any laptop, while production concerns are met by PostgreSQL without code changes.
- Vite SPA instead of Next.js (ADR-002) - the product is a private dashboard, not a content site; separation keeps the API the single security boundary.
- Analysis engine as pure functions (ADR-003) - later phases swap rule-based components for trained models behind identical signatures, so the API surface never changes.
- In-process rate limiting with an interface backing store (ADR-004) - correct for single-process deployment; Redis backing arrives with horizontal scaling in Phase 4.

Chapter 5 Database Design

5.1 Entity Overview

Phase 1 implements eight entities. The schema is deliberately drawn so the Phase 2-4 entities (stock, financial statement, news article, sentiment analysis, valuation, portfolio and holdings, reports, notifications, API usage) attach without reshaping existing tables.

Entity	Purpose	Key Attributes
User	Identity and role	email (unique, nullable), mobile (unique, nullable), password_hash, role, is_active, lockout counters

Entity	Purpose	Key Attributes
SessionToken	Revocable refresh session	token_hash (unique), user_agent, ip, expires_at, revoked_at, last_used_at
DisclaimerAcceptance	Versioned consent	version, ip_address, accepted_at
PasswordResetToken	Single-use reset	token_hash, expires_at, used_at
ExpenseCategory	System (user_id NULL) + custom categories	name, user_id
Expense	Core transaction	expense_date, amount(14,2), category FK, payment_method, merchant, tags, is_recurring, is_deleted
Income	Inflow records	income_date, amount, source, is_recurring, is_deleted
Budget	Spending limits	period, amount, category FK (NULL = overall), alert_thresholds (JSON), is_active
AuditLog	Append-only trail	action, entity, entity_id, ip, user_agent, meta (JSON)

5.2 Integrity and Indexing

Money columns use Numeric(14,2) - floating point is never used for currency. Soft deletion (is_deleted) preserves analytics history across corrections. Composite indexes serve the dominant access patterns: expenses (user_id, expense_date) for range analytics and (user_id, category_id) for distribution queries; incomes (user_id, income_date); session_tokens token_hash unique for refresh lookup; audit_logs (user_id, created_at) and (action) for admin timelines.

5.3 Design Decisions

- Email and mobile are both unique but nullable - the product allows signup with either; at least one is enforced at the service layer.
- Refresh tokens are stored only as HMAC-SHA256 digests keyed by the server secret; a database leak yields no usable credentials.
- System categories (user_id NULL) are shared and protected: they cannot be deleted, and categories in use by expenses or budgets are deletion-blocked at the service layer.

Chapter 6 API Design

6.1 Conventions

All endpoints are versioned under /api/v1. Responses are JSON with ISO-8601 timestamps and decimal-safe numbers. Errors return a machine-readable code with a human message. Interactive OpenAPI documentation is generated at /docs. The complete endpoint catalogue appears in Appendix A.

Code	HTTP	Meaning
DISCLAIMER_REQUIRED	403	Current disclaimer version not accepted
INVALID_CREDENTIALS	401	Bad login
ACCOUNT_LOCKED	423	Temporary lock after repeated failures
TOKEN_EXPIRED	401	Access or refresh token expired
VALIDATION_ERROR	422	Request failed schema validation
RATE_LIMITED	429	Too many requests

6.2 Rate Limiting

A sliding-window limiter protects the surface: authentication endpoints allow 10 requests per 5 minutes per IP, authenticated mutations 120 per minute per user, and reads 300 per minute per user. The limiter is an interface; Phase 4 swaps the backing store to Redis for multi-instance deployment.

6.3 Endpoint Groups

- **Auth -** 12 endpoints: registration, login, refresh, logout, profile view/update, password change, session list/revoke, password forgot/reset.
- **Finance -** 13 endpoints: expense CRUD with filters and pagination, category list/create/delete, income CRUD, budget CRUD with live status.
- **Analytics -** 7 endpoints: dashboard, summary, trends, categories, anomalies, insights, health-score.
- **Admin -** 5 endpoints: overview, user search/management, audit logs, login activity.

- **Meta -** 2 endpoints: liveness with database check, public current-disclaimer document.

Chapter 7 Security Architecture

7.1 Authentication

Passwords are hashed with bcrypt (cost 12); plaintext never touches storage or logs. Access tokens are HS256 JWTs with a 30-minute expiry and a per-token jti; refresh tokens are 48-byte random values stored only as keyed digests, expiring after 14 days. Every refresh rotates the session - the used token is revoked and replaced - so replayed refresh tokens fail fast. Five consecutive failures lock the account for 15 minutes; every attempt, successful or not, is audit-logged with IP and user agent.

7.2 Authorization and Least Privilege

Capability	User	Admin
Own financial data CRUD and analytics	Allowed	Denied
Admin panel, user management, audit trail	Denied (403)	Allowed
View any user's financial records	Denied	Denied by design

7.3 Input Handling and Transport Headers

Every request body is validated by typed Pydantic schemas (range, length, format - including an Indian mobile pattern). All database access uses SQLAlchemy bound parameters, eliminating string-built SQL. The API sets X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: no-referrer and Cache-Control: no-store on every response. Tokens travel exclusively in the Authorization header, removing the classic cookie CSRF vector.

7.4 Audit Trail

An append-only audit log records registrations, logins and failures, password changes and resets, disclaimer acceptances and every admin action, each with actor, IP, user agent and structured metadata. The admin console aggregates this trail into a login-activity timeline and raises a suspicious-activity flag when 24-hour failures exceed a threshold.

7.5 Known Phase 1 Limitations

- Password-reset tokens are returned in responses only in development; production requires SMTP wiring (tracked for Phase 2).
- Rate-limiter state is per-process - correct for the single-instance deployment, Redis-backed in Phase 4.
- Email/mobile verification is stubbed until a delivery provider is configured.

Chapter 8 AI/ML Methodology

8.1 Principles

Three principles govern every algorithmic component: no hallucinated finance (insights cite their numbers; insufficient data means silence); statistical transparency first, learned models later (Phase 2-3 models slot in behind identical function signatures); and explainability as a feature - scores decompose into weighted factors, flags carry their reference statistics.

8.2 Anomaly Detection (Z-Score)

Two detectors run over a bounded window. The transaction detector scores each expense against its category's distribution; the daily-total detector scores aggregate days against the user's baseline. Both require a minimum history before anything is flagged - 8 samples per category and 14 days overall - and both report the full reference statistics with every flag.

$$z = (x - \mu) / \sigma \qquad \textit{flag when } z > 2.5$$

where x is the candidate amount, and μ and σ are the mean and population standard deviation of the reference distribution. A flag's payload contains the amount, μ, σ, z, the sample size and a plain-language explanation, so the interface can show exactly why the transaction stood out.

8.3 Guarded Insight Rule Engine

Six deterministic rules convert aggregates into natural-language insights. Each rule has a data-sufficiency guard and fires only when its guard passes; each output carries an evidence object containing the exact figures used.

Rule	Fires When	Example Output
Category spike	category spend > 1.2 × previous month AND delta ≥ ₹500	“Food spending is up 185% - ₹2,198 vs ₹770 last month.”

Rule	Fires When	Example Output
Concentration	one category > 35% of period spend	“Housing represents 71% of spending this period.”
Recurring load	≥ 3 merchants recurring across ≥ 3 months	“Netflix, Spotify repeat monthly - about ₹798/month.”
Weekend pattern	weekend daily average > 1.3 × weekday and ≥ 14 days data	“Weekends cost 1.6× your weekday average.”
Savings trend	savings-rate change > ±5 pp with 2 months history	“Savings rate declined 8 points month-over-month.”
Budget pressure	utilisation crosses a configured threshold	“Budget 'Food' exceeded - 146% used.”

8.4 Explainable Financial Health Score

The score is a weighted sum over six factors. Factors lacking data are marked not_evaluated and their weight is redistributed proportionally across the evaluated ones - the model never invents inputs. The response reports every factor's weight, effective weight, factor score, contribution, a plain-language explanation and a suggestion when the factor scores below 80.

$$Score = \Sigma\ factorScore(i) \times effectiveWeight(i), \quad effectiveWeight(i) = w(i) / \Sigma\ w(evaluated)$$

Factor	Weight	Mapping to 0-100
Savings rate	0.25	Linear, full marks at 20% of income saved, zero at 0%
Expense-to-income	0.20	1.0 at ratio ≤ 0.5; 0 at ratio ≥ 1.0
Budget adherence	0.15	Full marks at ≤ 85% utilisation; zero at ≥ 120%
Recurring expense ratio	0.10	Full marks at ≤ 30% of spend; zero at ≥ 60%
Emergency fund	0.15	Months of average expenses covered; full marks at 6 months
Diversification	0.15	Portfolio-based; not_evaluated in Phase 1 (Phase 3 module)

Grades band the score: 85+ Excellent, 70+ Good, 55+ Fair, 40+ Needs Work, otherwise At Risk. Because contributions are additive and visible, the user can verify the arithmetic - a deliberate contrast with black-box scoring.

8.5 Planned Experiments (Phase 2-3 Evaluation Plan)

- **Experiment 1 -** Expense anomaly detection: statistical threshold vs Isolation Forest on injected outliers; metrics precision, recall, F1.
- **Experiment 2 -** Financial news sentiment: rule-based vs TF-IDF + linear SVM vs distilled transformer on a labelled sample; metrics accuracy, precision, recall, F1.
- **Experiment 3 -** Expense forecasting: moving average vs Holt linear vs ARIMA vs gradient boosting with walk-forward validation; metrics MAE, RMSE, MAPE.
- **Experiment 4 -** Explainable investment scoring: transparent weighted scoring vs opaque baseline; comprehensibility study plus factor ablation.

Chapter 9 Implementation

9.1 Repository Layout

The project is a monorepo: backend/ (FastAPI application - core, db, models, schemas, services, analytics, api/routes - plus tests and scripts), frontend/ (React + TypeScript application), docs/ (ten architecture documents, 01-10), docker-compose.yml, and a root README. The analysis engine lives in backend/app/analytics as four pure modules: aggregation, anomaly, insights and health_score.

9.2 Backend Implementation

The service layer encapsulates all orchestration. auth_service handles the credential lifecycle including lockout and rotation; expense_service covers expenses, categories, incomes and budgets with live utilisation computation; analytics_service assembles bounded windows of user data and feeds the pure engine; admin_service exposes operational aggregates only; audit_service appends the immutable trail. Startup initialises the schema, ensures the twelve system categories, and waits for database readiness.

9.3 Frontend Implementation

The SPA implements a glassmorphism design system on an ambient aurora background with a dark/light-capable token set. A thin typed API client performs silent JWT refresh-and-retry on 401. Route guards enforce authentication; the disclaimer gate precedes registration; role gating exposes the admin console to admins only. Charts are Recharts-based (donut, bar, line, area) with muted axes and direct value labels.

9.4 Interface Walkthrough

The following captures are taken from the running containerised deployment (production build, nginx on port 8080, PostgreSQL backend) with a seeded demo account carrying two months of transactions.

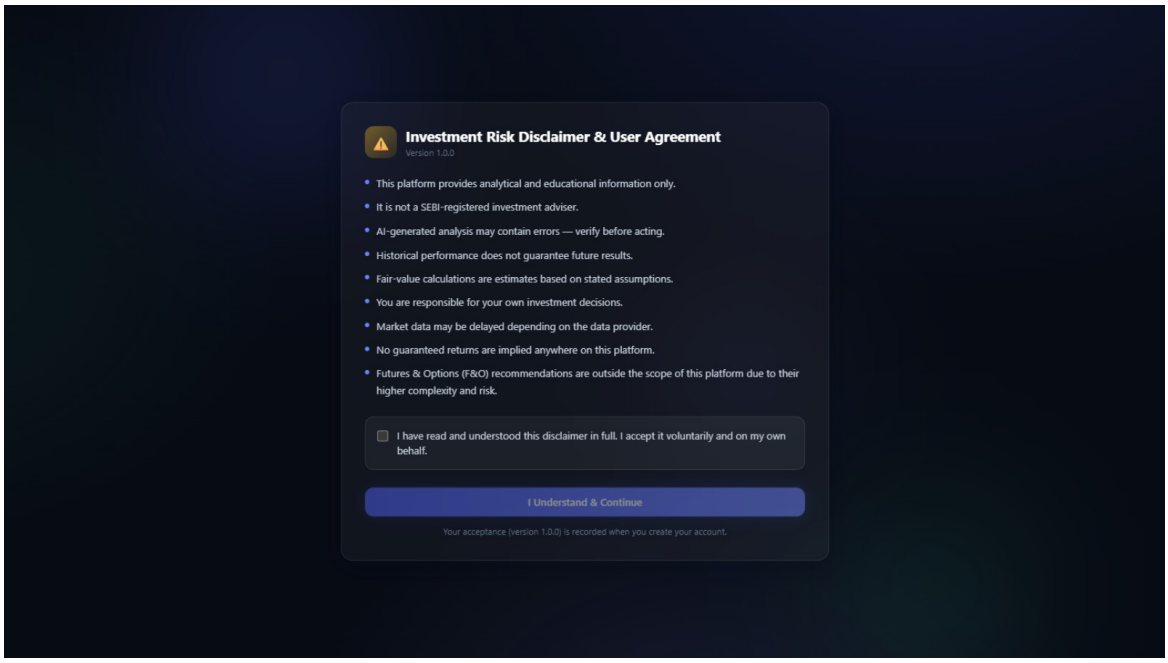


Figure 1: Versioned investment risk disclaimer - acknowledgement is mandatory and recorded server-side

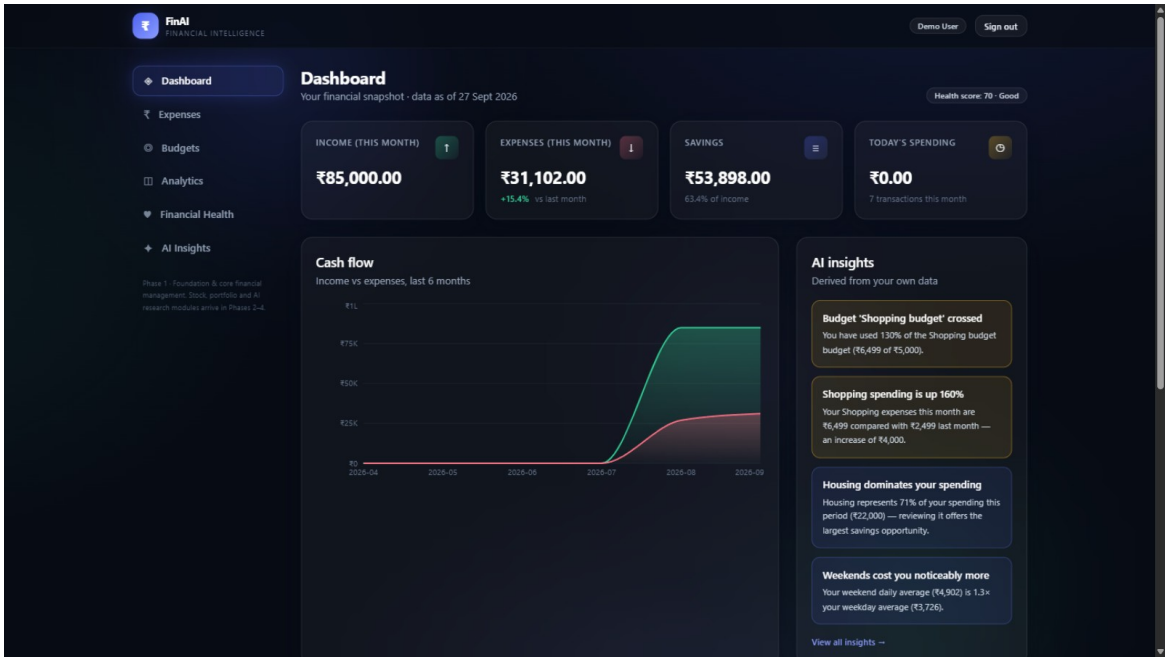


Figure 2: Dashboard - financial snapshot, six-month cash-flow, evidence-backed AI insights, budget utilisation

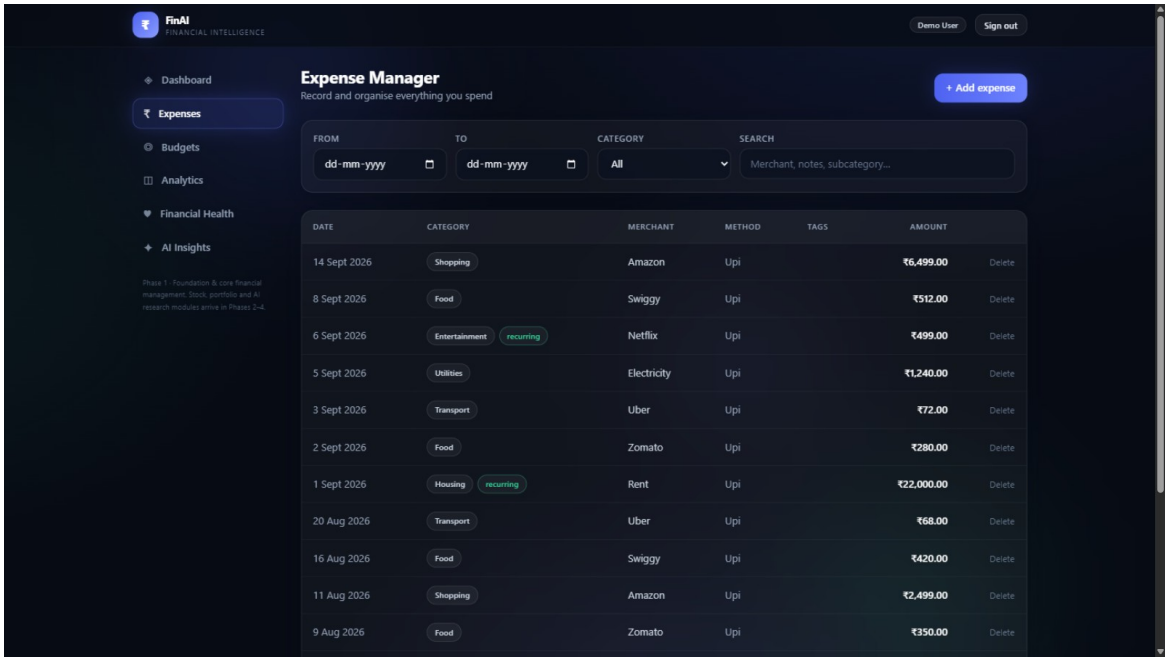


Figure 3: Expense manager - filters, search, pagination, custom categories, recurring and tag support

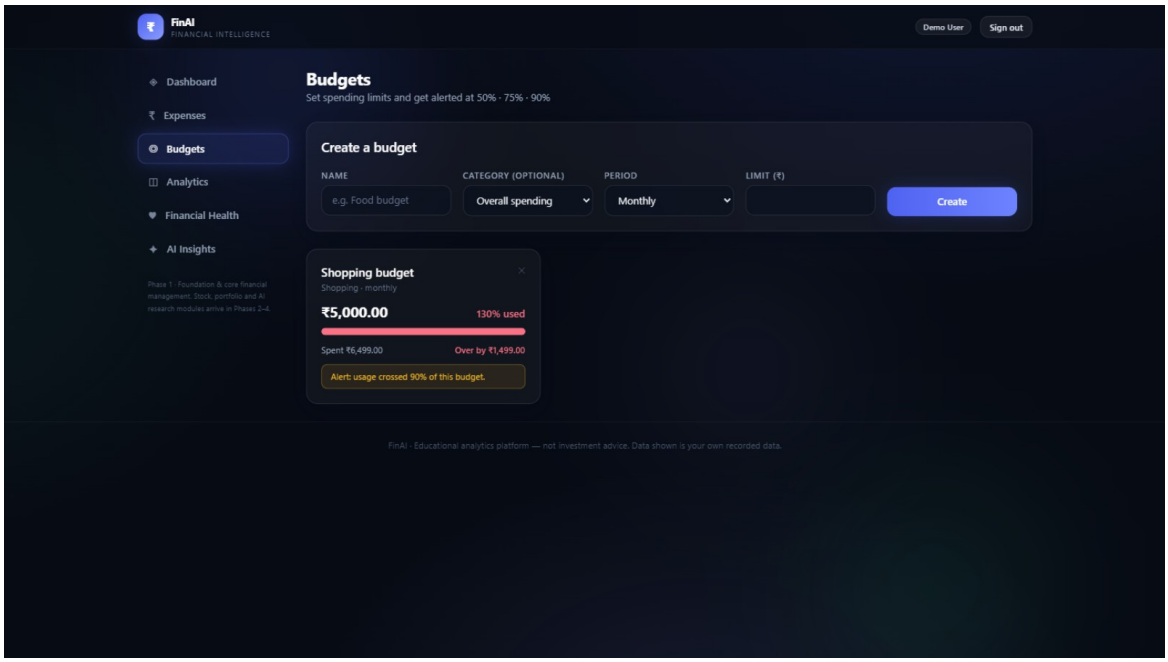


Figure 4: Budgets - live utilisation bars with threshold alerts

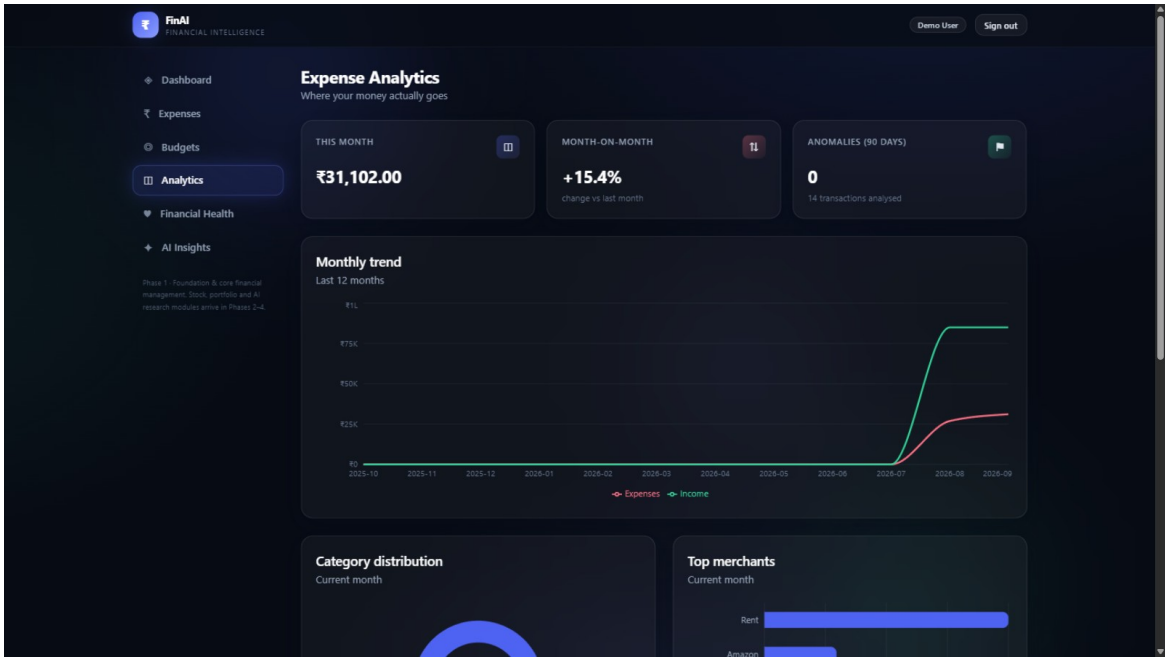


Figure 5: Expense analytics - monthly trend, category distribution, top merchants, anomaly flags



Figure 6: Financial health - transparent score with per-factor weights, contributions and suggestions

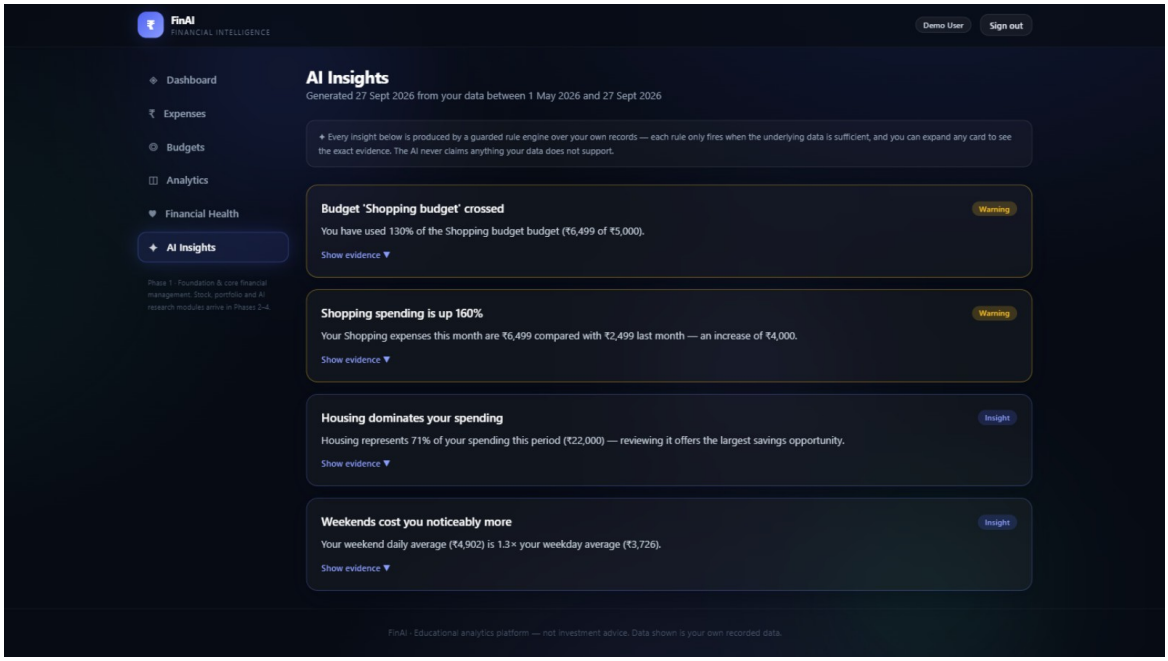


Figure 7: AI insights feed - each card exposes its underlying evidence

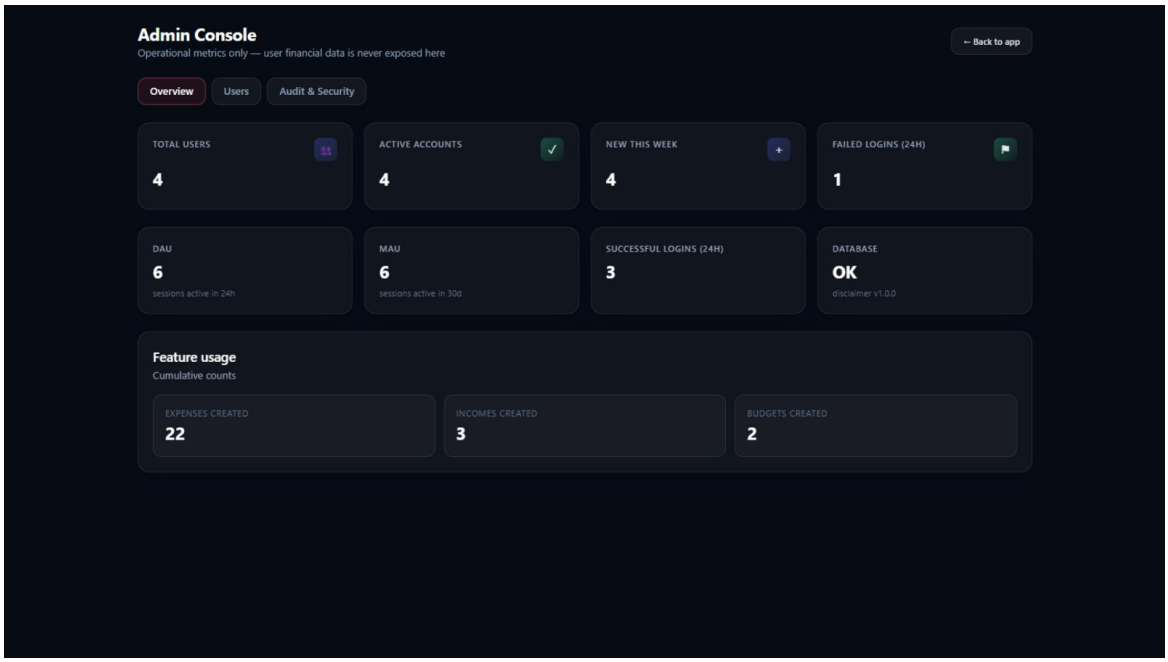


Figure 8: Admin console - operational metrics only; user financial data is never exposed

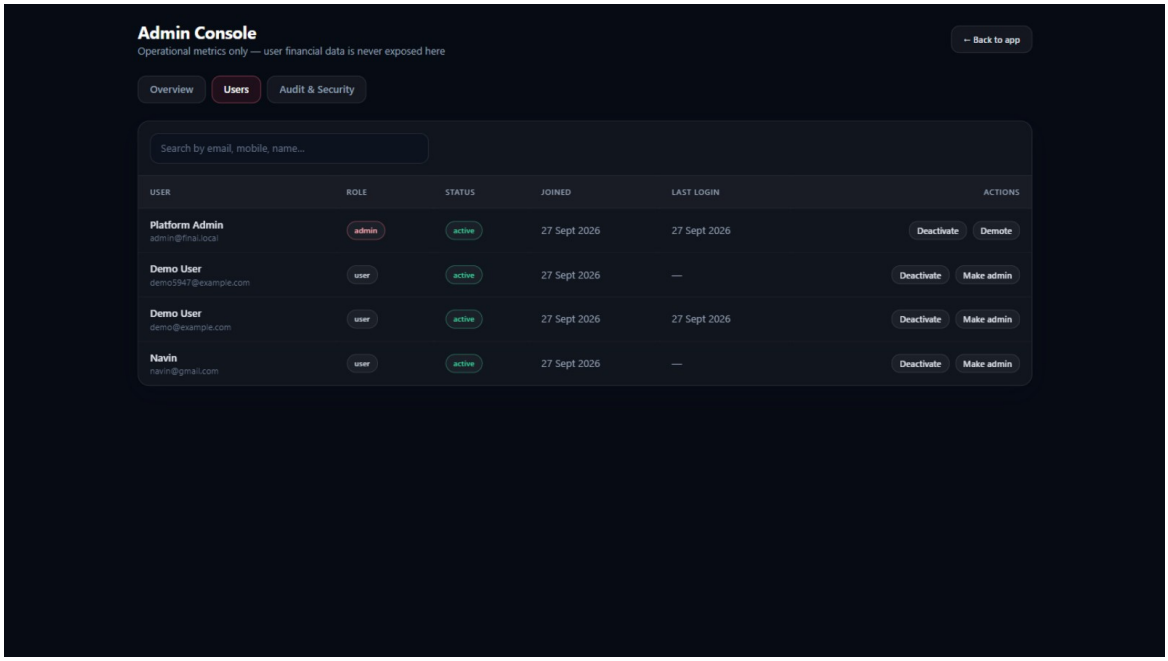


Figure 9: Admin user management - activation, role changes, audit trail and login-activity views

Chapter 10 Testing and Results

10.1 Test Strategy

The suite follows the test pyramid. Unit tests exercise the pure analytics engine without I/O; integration tests drive the real application over an isolated SQLite database per test, seeding exclusively through the public API so behaviour - not implementation - is verified. Financial assertions use approximate comparison throughout.

10.2 Test Inventory and Outcome

Suite	Coverage	Tests	Result
Analytics engine (unit)	Aggregation, MoM/YoY, distribution, weekend split, recurring detection, z-score guards, all six insight rules, score weight redistribution	17	Pass

Suite	Coverage	Tests	Result
Authentication (integration)	Register (email/mobile/duplicate), disclaimer gate, lockout, refresh rotation, logout revocation, password change/reset, sessions, RBAC denial	13	Pass
Expenses & categories (integration)	CRUD, validation, filters, pagination, cross-user isolation, category lifecycle guards, income flow	7	Pass
Budgets, analytics & admin (integration)	Budget math and alerts, dashboard/trends/categories payloads, anomaly and insight shapes, health factors, admin flows, self-deactivation guard	10	Pass
Total		47	All pass

10.3 End-to-End Verification

A scripted journey exercises the deployed system exactly as a demonstration would: disclaimer fetch, registration, income and expense creation, budget creation, listing, dashboard and analytics retrieval, insight generation, health scoring, RBAC refusal of a non-admin on admin endpoints, admin login and administrative views. The journey passes in full against both the development and the containerised PostgreSQL deployment, with the demo dataset producing two evidence-backed insights, a 94.1% budget utilisation triggering the 50/75/90 alert chain, and a health score of 95 (Excellent) with five of six factors evaluated - diversification correctly reported as not evaluated.

10.4 Observations

- Data-sufficiency guards behaved as designed: with sparse history the anomaly detector stays silent instead of fabricating flags.
- Weight redistribution kept the health score arithmetically consistent when the diversification factor was unavailable.

- Refresh-token rotation invalidated replayed tokens immediately in every attempt, and revocation propagated to logout within the same session.

Chapter 11 Deployment

11.1 Development Setup

The backend runs from a virtual environment (requirements.txt) with a .env file supplying SECRET_KEY, DATABASE_URL, CORS origins and the disclaimer version; a seed script creates system categories and the initial administrator. The frontend runs under the Vite dev server, which proxies /api to the backend so the browser sees a single origin.

11.2 Containerised Stack

Service	Image / Build	Port	Role
postgres	postgres:16-alpine	5432	Primary datastore (volume-persisted), health-checked
redis	redis:7-alpine	6379	Provisioned for Phase 2+ caching and job queues
api	Built (python:3.11-slim)	8000	FastAPI application; waits for database readiness at startup
web	Built (node build + nginx)	8080	Production SPA bundle; reverse-proxies /api to the API container

Secrets are injected via environment; .env files are git-ignored and .env.example documents every key. The API image excludes tests, virtual environments and local databases through .dockerignore, and no API key or secret ever reaches the frontend bundle.

11.3 Operational Behaviour

The startup sequence demonstrates the reliability requirement: on first boot the API container encountered a momentarily refusing database and, rather than crashing permanently, waited for readiness and completed initialisation - schema creation, category seeding and service start - without manual intervention. A /healthz endpoint reports application and database status for orchestrators.

Chapter 12 Limitations and Future Work

12.1 Current Limitations

- Investment analytics are not yet implemented; market-data panels correctly display 'Data unavailable' rather than fabricated values.
- The rate limiter and password-reset delivery carry the Phase 1 constraints documented in Chapter 7.
- Forecasting and learned models await sufficient historical data by design - guards refuse to run on thin data.

12.2 Phase 2 - Investment Analytics

Provider abstraction (FinancialDataProvider interface) with a first provider and response caching; stock analyzer covering fundamentals, statement interpretation and technical indicators (SMA/EMA, RSI, MACD, Bollinger, drawdown); news ingestion from legitimate APIs; the Experiment 2 sentiment comparison; a valuation engine with DCF (editable assumptions and sensitivity grids), PE, PEG, EV/EBITDA and DDM; explainable stock scoring; PDF research reports; Alembic migrations begin.

12.3 Phase 3 - Portfolio Intelligence and Financial Planning

Portfolio analyzer with holdings, allocation and P/L; risk metrics (volatility, maximum drawdown, HHI concentration, beta); benchmark comparison and stress-test scenarios; net-worth tracking; goals engine; FD, SIP, lumpsum and compound-growth calculators; retirement planner and inflation analysis; the Experiment 1 and 3 evaluations.

12.4 Phase 4 - Intelligent Financial Platform

Retrieval-augmented research assistant with mandatory citations; LLM summarisation over retrieved context only; background jobs on Redis (ingestion, sentiment batches, report generation); caching and observability; expanded admin analytics; security hardening; Playwright end-to-end suite; performance testing and deployment guide; dissertation documentation freeze.

Chapter 13 Conclusion

Phase 1 of FinAI demonstrates that an explainable, data-grounded approach to personal financial intelligence is achievable with production engineering discipline. The delivered increment comprises a hardened authentication and authorization core governed by a versioned disclaimer, complete expense and budget management, a multi-granularity analytics engine, and an AI layer - anomaly detection, guarded insights, transparent health

scoring - whose every output is either supported by the user's data or withheld. The implementation is verified by 47 passing tests and a scripted end-to-end journey against a containerised, production-shaped deployment.

The phase plan keeps the academic focus on engineering: each subsequent phase adds a well-bounded capability - investment analytics, portfolio intelligence, and finally a retrieval-grounded assistant - behind interfaces that already exist. The result is a platform fit for its four internal reviews, the final demonstration, and the dissertation's methodological chapters.

References

[1] Ramirez, E. and team, FastAPI: Modern, Fast Web Framework for Building APIs with Python, https://fastapi.tiangolo.com (accessed September 2026).

[2] SQLAlchemy 2.0 Documentation, SQLAlchemy authors, https://docs.sqlalchemy.org (accessed September 2026).

[3] React - A JavaScript Library for Building User Interfaces, Meta Open Source, https://react.dev (accessed September 2026).

[4] Recharts: A Composable Charting Library Built on React Components, https://recharts.org (accessed September 2026).

[5] Provos, N. and Mazieres, D., A Future-Adaptable Password Scheme (bcrypt), Proceedings of the USENIX Annual Technical Conference, 1999.

[6] Jones, M., Bradley, J. and Sakimura, N., RFC 7519: JSON Web Token (JWT), IETF, 2015.

[7] Liu, F. T., Ting, K. M. and Zhou, Z.-H., Isolation Forest, Proceedings of the IEEE International Conference on Data Mining, 2008.

[8] Hyndman, R. J. and Athanasopoulos, G., Forecasting: Principles and Practice, 3rd ed., OTexts, 2021.

[9] PostgreSQL 16 Documentation, The PostgreSQL Global Development Group, https://www.postgresql.org/docs/ (accessed September 2026).

[10] Graham, B., The Intelligent Investor, Harper & Brothers, 1949 (valuation principles referenced for Phase 2).

[11] Docker Documentation, Docker Inc., https://docs.docker.com (accessed September 2026).

[12] Pydantic Documentation - Data Validation Using Python Type Hints, https://docs.pydantic.dev (accessed September 2026).

Appendix A REST API Catalogue (Phase 1)

Group	Method & Path	Purpose
Auth	POST /auth/register	Create account with disclaimer acceptance
Auth	POST /auth/login	Authenticate; lockout after 5 failures
Auth	POST /auth/refresh	Rotate access + refresh tokens
Auth	POST /auth/logout	Revoke the presented refresh session
Auth	GET/PATCH /auth/me	Profile view / update
Auth	POST /auth/change-password	Change password (requires current)
Auth	GET/DELETE /auth/sessions	List active sessions / revoke one
Auth	POST /auth/password/forgot | /reset	Dev-visible token; single-use reset, sessions revoked
Finance	GET/POST /expenses; GET/PATCH/DELETE /expenses/{id}	Expense list (filters, search, paging) and CRUD
Finance	GET/POST /categories; DELETE /categories/{id}	System + custom categories with usage guards
Finance	GET/POST /incomes; PATCH/DELETE /incomes/{id}	Income CRUD
Finance	GET/POST /budgets; PATCH/DELETE /budgets/{id}	Budgets with live utilisation and alerts
Analytics	GET /analytics/dashboard	One-shot dashboard payload
Analytics	GET /analytics/summary | trends | categories	Period totals, monthly series, distributions
Analytics	GET /analytics/anomalies | insights | health-score	AI outputs with evidence
Admin	GET /admin/overview	Users, DAU/MAU, feature usage, system health
Admin	GET /admin/users; PATCH /admin/users/{id}	Search and manage accounts (activate, role)
Admin	GET /admin/audit-logs | login-activity	Security trail and login timeline

Group	Method & Path	Purpose
Meta	GET /healthz; GET /config/disclaimer	Liveness with DB check; public disclaimer

Appendix B Risk Register

#	Risk	L	I	Mitigation
1	Data provider changes terms or breaks	H	H	Provider abstraction; cached history; never fabricate missing data
2	External API quota exhaustion	M	M	Server-side caching with freshness; staggered ingestion (P2)
3	LLM hallucination in research answers	M	H	RAG with mandatory citations; refuse when retrieval is empty
4	Weak sentiment accuracy on finance text	M	M	Three-model comparison experiment before selection; rule-based fallback
5	Scope creep	H	H	Strict four-phase plan; demo-first increments
6	Insufficient history for forecasting	M	M	Minimum-data guards refuse to run
7	Secret leakage	L	H	Backend-only env; git-ignored .env; no keys in frontend

#	Risk	L	I	Mitigation
8	Credential stuffing	M	M	Rate limiting + lockout + audit alerts (implemented)
9	Password reset without SMTP	M	M	Dev-only token exposure; SMTP tracked for P2
10	SQLite to PostgreSQL drift	L	M	Portable types; compose ships PostgreSQL; tested
11	Performance on large histories	L	M	Indexes, pagination, bounded windows; profiling in P4
12	Single-process rate-limit state	L	L	Interface limiter; Redis backing in P4
13	Platform mistaken for advice	M	H	Disclaimer gate; analysis-not-advice framing throughout
14	Timeline pressure across reviews	M	M	Milestones M1-M8; riskiest integration early in P2

Appendix C Four-Phase Roadmap and Review Plan

Phase	Focus	Review Emphasis	Status
Phase 1	Foundation: architecture, auth, expenses, budgets, analytics, guarded AI insights, health score, admin	Can we build it? Architecture, stack, ER, security, initial AI	Complete

Phase	Focus	Review Emphasis	Status
Phase 2	Investment analytics: providers, stock analyzer, valuation, sentiment, PDF reports	Can we analyse financial data intelligently? DCF math, NLP, explainability	Planned
Phase 3	Portfolio intelligence and planning: holdings, risk, stress tests, calculators, goals	Can we integrate portfolio intelligence and planning?	Planned
Phase 4	Intelligent platform: RAG assistant, jobs, caching, observability, hardening, e2e	Can we demonstrate a complete AI-driven platform?	Planned

Final demonstration flow: disclaimer, registration, dashboard, income and expense recording, analytics, AI insight, stock analysis, valuation, sentiment, research report, portfolio and stress test, calculators, PDF report, and admin monitoring with the audit trail - each step mapping one-to-one onto a delivered or planned module.